```
// SPDX-License-Identifier: MIT pragma solidity ^0.8.20;

contract EVoting { address public owner;

bool public electionStarted; bool public electionEnded;

struct Candidate { uint256 id; string name;

uint256 voteCount;

}

uint256 public candidateCount;

mapping(uint256 => Candidate) public candidates; mapping(address => bool) public
hasVoted;

event CandidateAdded(uint256 indexed id, string name);

event VoteCast(address indexed voter, uint256 indexed candidateId); event
ElectionStarted();

event ElectionEnded();

modifier onlyOwner() {

require(msg.sender == owner, "Only owner can perform this action");

_;

}

modifier duringVoting() {

require(electionStarted && !electionEnded, "Voting is not active");

_;

}

constructor() {
```

```
owner = msg.sender;

}

function addCandidate(string calldata name) external onlyOwner { require(!electionStarted, "Cannot add after election starts"); require(bytes(name).length > 0, "Candidate name required");

candidateCount++; candidates[candidateCount] = Candidate(

candidateCount, name,

0

);

emit CandidateAdded(candidateCount, name);

}

function startElection() external onlyOwner { require(candidateCount >= 2, "Add at least two candidates"); require(!electionStarted, "Election already started");

electionStarted = true; emit ElectionStarted();

}

function vote(uint256 candidateId) external duringVoting { require(!hasVoted[msg.sender], "Address has already voted"); require(candidateId > 0 && candidateId <= candidateCount, "Invalid candidate");

hasVoted[msg.sender] = true; candidates[candidateId].voteCount++;

emit VoteCast(msg.sender, candidateId);
```

}

function endElection() external onlyOwner { require(electionStarted, "Election not started"); require(!electionEnded, "Election already ended");

electionEnded = true; emit ElectionEnded();

}

function getCandidate(uint256 candidateId) external

view

returns (uint256 id, string memory name, uint256 voteCount)

{

Candidate memory c = candidates[candidateId]; return (c.id, c.name, c.voteCount);

}

}

Output: Smart Contract – Blockchain E-Voting dApp

Your Solidity smart contract `EVoting` allows the owner to manage an election and users to vote for candidates.

Output in Remix IDE

1. Deploy the contract

```
EVoting
Status: Transaction successful
Deployed at: 0x1234...abcd
```

2. Add candidates

Call `addCandidate("Alice")` and `addCandidate("Bob")`.

```
Candidate 1: Alice
Candidate 2: Bob
candidateCount: 2
```

3. Start the election

Call `startElection()` from the owner's account.

```
electionStarted: true
electionEnded: false
```

4. Cast votes

Call `vote(1)` to vote for Alice and `vote(2)` to vote for Bob, using different accounts.

```
Alice — Votes: 1
Bob   — Votes: 1
```

5. Check voting restrictions

- A user who has already voted cannot vote again.
- Invalid candidate IDs are rejected.
- Only the owner can add candidates, start the election, and end it.
- Voting is allowed only while the election is active.

6. End the election

Call `endElection()` from the owner's account.

```
electionStarted: true
electionEnded: true
```

Simple Web dApp

Save the following as index.html. Replace CONTRACT_ADDRESS with the deployed contract address and ABI with the ABI generated by Remix.

```
<!DOCTYPE html>

<html>

<head>

<meta charset="UTF-8">

<title>Blockchain E-Voting dApp</title>

</head>

<body>

<h1>E-Voting dApp</h1>

<button onclick="connectWallet()">Connect MetaMask</button>

<p id="account"></p>


<h2>Candidates</h2>

<div id="candidates"></div>


<script type="module"> import { ethers } from

"https://cdn.jsdelivr.net/npm/ethers@6.13.5/+esm";


const CONTRACT_ADDRESS = "PASTE_DEPLOYED_CONTRACT_ADDRESS";

const ABI = [

"function candidateCount() view returns (uint256)",

"function candidates(uint256) view returns (uint256,string,uint256)", "function vote(uint256 candidateId)",

"function hasVoted(address) view returns (bool)", "function electionStarted() view returns (bool)", "function electionEnded() view returns (bool)"

];
```

```
let provider, signer, contract;

window.connectWallet = async function () { if (!window.ethereum) {
alert("Please install MetaMask."); return;
}

provider = new ethers.BrowserProvider(window.ethereum); await
provider.send("eth_requestAccounts", []);
signer = await provider.getSigner();

document.getElementById("account").innerText = "Connected: " + await signer.getAddress();

contract = new ethers.Contract( CONTRACT_ADDRESS, ABI, signer
);

await loadCandidates();
};

async function loadCandidates() {
const count = Number(await contract.candidateCount()); const box =
document.getElementById("candidates"); box.innerHTML = "";

for (let i = 1; i <= count; i++) {
const c = await contract.candidates(i);

const row = document.createElement("p"); row.innerHTML =
c[0] + ". " + c[1] +
" | Votes: " + c[2] +
' <button data-id="' + c[0] + '">Vote</button>';
```

```
row.querySelector("button").onclick = async () => { try {
const tx = await contract.vote(c[0]); await tx.wait();
alert("Vote recorded.");

await loadCandidates();
} catch (err) {
alert(err.shortMessage || err.message);
}
};

box.appendChild(row);
}
}
</script>
</body>
</html>
```

